

Quick answer key to Asymptotics

ChatGPT (with inputs by Arjun)

24 February

Use the table of contents below to skip to a specific part without seeing spoilers to the other parts.

I just used ChatGPT to write this one quickly. ChatGPT can make mistakes, so if you spot anything that's wrong, flag me to ask.

Contents

1	Solving Recurrences	2
1.1	1. $T(n) = T(\sqrt{n}) + O(n)$	2
1.2	2. $T(n) = 6T(n/3) + O(n^2)$	3
1.3	3. $T(n) = 4T(n/5) + O(n)$	4
1.4	4. $T(n) = 3T(\sqrt{n}) + \log n$	5
2	Asymptotic Comparisons	6
2.1	1. $f(n) = n \log n, g(n) = 10n \log(10n)$	6
2.2	2. $f(n) = n^{1.01}, g(n) = n \log^2 n$	7
2.3	3. $f(n) = (\log n)^{\log n}, g(n) = \frac{n}{\log n}$	8
2.4	4. $f(n) = \log(n!), g(n) = n \log n$	9
2.5	5. $f(n) = \binom{n}{k}, g(n) = n^k$	10
2.6	6. $f(n) = n, g(n) = n^{1+\sin n}$	11
3	Harder Recurrences	12
3.1	1. $T(n) = \sum_{i=1}^{n-1} T(i) + 1$	12
3.2	2. $T(n) = T(7n/10) + T(n/5) + O(n)$	13
3.3	3. $T(n) = T(\log n) + n$	14
3.4	4. $T(n) = T(n - \sqrt{n}) + 1$	15
3.5	5. $T(n) = T(n - 1) + n^k$	16

1 Solving Recurrences

1.1 1. $T(n) = T(\sqrt{n}) + O(n)$

This recurrence is not in standard Master Theorem form because the subproblem size is $\sqrt{n}$ rather than n/b .

Let us perform a change of variables.

Set

$$n = 2^m.$$

Define

$$S(m) = T(2^m).$$

Then:

$$T(\sqrt{n}) = T(2^{m/2}) = S(m/2).$$

Thus the recurrence becomes

$$S(m) = S(m/2) + O(2^m).$$

Now this is in Master Theorem form:

$$S(m) = aS(m/b) + f(m)$$

with:

$$a = 1, \quad b = 2, \quad f(m) = \Theta(2^m).$$

Compute:

$$m^{\log_b a} = m^{\log_2 1} = m^0 = 1.$$

We compare $f(m) = \Theta(2^m)$ to 1.

Clearly 2^m grows exponentially faster than any polynomial. Thus we are in Master Theorem Case 3.

Therefore:

$$S(m) = \Theta(2^m).$$

Returning to $n = 2^m$, we conclude:

$$T(n) = \Theta(n).$$

$T(n) = \Theta(n)$

1.2 2. $T(n) = 6T(n/3) + O(n^2)$

Now this is exactly in Master Theorem form:

$$a = 6, \quad b = 3, \quad f(n) = \Theta(n^2).$$

Compute:

$$n^{\log_b a} = n^{\log_3 6}.$$

Note:

$$\log_3 6 = \frac{\ln 6}{\ln 3} > 1.$$

Numerically,

$$\log_3 6 \approx 1.63.$$

So:

$$n^{\log_3 6} = \Theta(n^{1.63}).$$

Compare $f(n) = \Theta(n^2)$.

Since $2 > 1.63$, we are in Master Theorem Case 3.

Thus:

$$T(n) = \Theta(n^2).$$

$T(n) = \Theta(n^2)$

1.3 3. $T(n) = 4T(n/5) + O(n)$

Here:

$$a = 4, \quad b = 5, \quad f(n) = \Theta(n).$$

Compute:

$$n^{\log_b a} = n^{\log_5 4}.$$

Since $4 < 5$, we have:

$$\log_5 4 < 1.$$

Thus:

$$n^{\log_5 4} = \Theta(n^c) \quad \text{with } c < 1.$$

Now compare $f(n) = \Theta(n)$.

Since n grows polynomially faster than n^c , we are again in Case 3.

Hence:

$$T(n) = \Theta(n).$$

$$\boxed{T(n) = \Theta(n)}$$

1.4 4. $T(n) = 3T(\sqrt{n}) + \log n$

Again substitute $n = 2^m$ and define $S(m) = T(2^m)$.

Then:

$$S(m) = 3S(m/2) + m.$$

Master Theorem form:

$$a = 3, \quad b = 2, \quad f(m) = \Theta(m).$$

Compute:

$$m^{\log_2 3}.$$

Since $\log_2 3 > 1$, this is polynomial with exponent about 1.585.

Compare $f(m) = \Theta(m)$.

Since $1 < \log_2 3$, we are in Case 1.

Therefore:

$$S(m) = \Theta(m^{\log_2 3}).$$

Return to $n = 2^m$:

$$m = \log n.$$

Thus:

$$T(n) = \Theta((\log n)^{\log_2 3}).$$

$$T(n) = \Theta((\log n)^{\log_2 3})$$

2 Asymptotic Comparisons

2.1 1. $f(n) = n \log n$, $g(n) = 10n \log(10n)$

Observe:

$$\log(10n) = \log 10 + \log n.$$

Thus:

$$g(n) = 10n(\log n + \log 10).$$

As $n \rightarrow \infty$, $\log 10$ is constant.

Hence:

$$g(n) = \Theta(n \log n).$$

Therefore:

$$f(n) = \Theta(g(n)).$$

2.2 2. $f(n) = n^{1.01}$, $g(n) = n \log^2 n$

Canceling an n on both sides and considering $f^*(n) = n^{0.1}$ and $g^*(n) = \log^2 n$.

Making the usual substitution $n = 2^k$, $g^*(n) = k^2$ while $f^*(n) = 2^{(k/10)}$.

Thus, $g^*(n) = O(f^*(n))$

Thus, $g(n) = O(f(n))$.

Note: AI partially failed at solving as it basically claimed that the answer is the answer.

2.3 3. $f(n) = (\log n)^{\log n}$, $g(n) = \frac{n}{\log n}$

Take logarithms:

$$\log f(n) = \log n \cdot \log \log n.$$

While:

$$\log g(n) = \log n - \log \log n.$$

Since:

$$\log n \log \log n \gg \log n,$$

we conclude:

$$f(n) \gg g(n).$$

Thus:

$$g(n) = O(f(n)).$$

2.4 4. $f(n) = \log(n!)$, $g(n) = n \log n$

We bound:

Upper bound:

$$n! \leq n^n \Rightarrow \log(n!) \leq n \log n.$$

Lower bound:

$$\begin{aligned} n! &\geq n \times (n-1) \times (n-2) \times \cdots \times n/2 \\ &\geq n/2 \times n/2 \times n/2 \times \cdots \times n/2 \\ &\geq (n/2)^{n/2} \\ \Rightarrow n! &\geq (n/2)^{n/2} \\ \Rightarrow \log(n!) &\geq n/2 \log(n/2) \\ &= n/2(\log n - \log 2) \\ &= n/2 \log n - n/2 \log(2) \\ \Rightarrow \log(n!) &\geq O(n \log n) \end{aligned}$$

Thus:

$$\log(n!) = \Theta(n \log n).$$

Note: AI messed this up and just claimed theorem. Although, more prompting got this answer.

2.5 5. $f(n) = \binom{n}{k}$, $g(n) = n^k$

We start with:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{n(n-1)(n-2)\dots(n-k+1)}{k!}.$$

Upper Bound:

Each term in the numerator satisfies

$$n - i \leq n \quad \text{for } 0 \leq i \leq k - 1.$$

Thus,

$$n(n-1)\dots(n-k+1) \leq n^k.$$

Dividing by $k!$:

$$\binom{n}{k} \leq \frac{n^k}{k!}.$$

Since $k!$ is a constant (because k is fixed),

$$\binom{n}{k} = O(n^k).$$

Lower Bound:

For sufficiently large $n > 2k$, we have

$$n - i \geq \frac{n}{2} \quad \text{for all } 0 \leq i \leq k - 1.$$

(This holds whenever $n \geq 2k$.)

Thus,

$$n(n-1)\dots(n-k+1) \geq \left(\frac{n}{2}\right)^k.$$

Dividing by $k!$:

$$\binom{n}{k} \geq \frac{1}{k!} \left(\frac{n}{2}\right)^k = \frac{1}{k!2^k} n^k.$$

Since $\frac{1}{k!2^k}$ is a constant,

$$\binom{n}{k} = \Omega(n^k).$$

Thus, $\binom{n}{k} = \Theta(n^k)$.

Note: AI messed this up and just claimed theorem. Although, more prompting got this answer.

2.6 6. $f(n) = n$, $g(n) = n^{1+\sin n}$

Since $\sin n \in [-1, 1]$,

$$g(n) \in [n^0, n^2].$$

Thus infinitely often:

$$g(n) = n^2 \quad \text{and} \quad g(n) = 1.$$

Therefore neither function bounds the other asymptotically.

Neither $f = O(g)$ nor $g = O(f)$.

3 Harder Recurrences

Most of these had to be edited quite a bit as GPT failed at them quite a bit.

3.1 1. $T(n) = \sum_{i=1}^{n-1} T(i) + 1$

Consider $T(n) - T(n-1) = (\sum_{i=1}^{n-1} T(i) + 1) - (\sum_{i=1}^{n-2} T(i) + 1) = T(n-1)$

Thus, $T(n) = 2T(n-1)$.

Thus, $T(n) = 2^n T(1) \implies T(n) = O(n)$.

Note: This was not hard. Somehow, GPT failed at it and just used induction after saying "notice the pattern".

3.2 2. $T(n) = T(7n/10) + T(n/5) + O(n)$

As

$$\frac{7}{10} + \frac{1}{5} = 0.7 + 0.2 = 0.9 < 1.$$

Thus, $T(n) = T(7n/10) + T(n/5) + O(n) = T(49n/100) + T(n/25) + O(9n/10) + O(n)$. The sum is geometrically decreasing, hence we make the guess: $T(n) \leq 10n = O(n)$.

(B) For $n = 1$, we have $T(1) = 1 < 10$.

(S) Let $T(k) \leq 10k$ for $k < n$. We will show $T(n) \leq 10n$.

$$\begin{aligned} T(n) &= T(7n/10) + T(n/5) + n \\ &\leq 10 \frac{7n}{10} + 10 \frac{n}{5} + n \\ &= 10n \end{aligned}$$

Hence, by induction, we are done!

3.3 3. $T(n) = T(\log n) + n$

Expand:

$$T(n) = n + \log n + \log \log n + \dots$$

This series is dominated by the first term.

Thus, we conclude by induction:

$$\boxed{T(n) = \Theta(n)}$$

3.4 4. $T(n) = T(n - \sqrt{n}) + 1$

Our basic question is that "how many times do we need to subtract $\sqrt{n}$ before the number reaches some constant."

So what do we do? One idea is to try out some values and then guess. The other idea is to consider a continuous extension.

$$\begin{aligned}\frac{dn}{dt} &= -\sqrt{n} \\ -\frac{dn}{\sqrt{n}} &= dt \\ \int -\frac{dn}{\sqrt{n}} &= \int dt \\ t &= O(\sqrt{n})\end{aligned}$$

And we now complete by induction!

3.5 5. $T(n) = T(n-1) + n^k$

Unroll:

$$T(n) = \sum_{i=1}^n i^k.$$

If you remember the forms for $k = 1, 2, 3$ that is $\frac{n(n+1)}{2}$, $\frac{n(n+1)(2n+1)}{6}$ and $(\frac{n(n+1)}{2})^2$ respectively; you could reasonably guess that this extends to the general case $T(n) = \Theta(n^{k+1})$ and induct away.

The "formal, non-guess way" is we compare to integral:

$$\int_1^n x^k dx = \Theta(n^{k+1}).$$

Thus, we conclude by induction

$$\boxed{T(n) = \Theta(n^{k+1})}$$